

第一次实践环节：MNIST 手写数字识别实验手册

0. 使用本手册前先确认

本手册默认你已经阅读或完成同目录下的：

[从0开始Python快速入门-面向深度学习实践.md](#)

因此，本手册不再重复讲：

1. Miniconda 怎么安装。
 2. VS Code 怎么安装。
 3. Conda 环境怎么创建。
 4. Python 变量、列表、循环、函数、类等基础语法。
 5. NumPy、Matplotlib、Path 的基础用法。
-

1. 本次实验要做什么

MNIST 是一个手写数字数据集，每张图片都是一个手写数字，类别为 0-9。

本次任务是：

输入：一张 28 x 28 的灰度手写数字图片
输出：模型判断这张图片是 0-9 中的哪一个数字

我们使用最容易理解的 MLP，也就是全连接神经网络。

2. 本次实验的学习目标

完成本实验后，你应该能理解：

1. 什么是训练集和测试集。
2. 什么是 batch。
3. MNIST 图片为什么可以从 28 x 28 展平成 784 个数字。
4. MLP 全连接网络的基本结构。
5. 一次训练循环包括哪些步骤。
6. loss、accuracy 分别代表什么。
7. PyTorch 中 `model`、`loss_fn`、`optimizer` 分别负责什么。
8. 如何保存训练日志和预测图片。

9. 如何通过修改超参数观察实验结果变化。

3. 最小检查

请先激活课程环境。假设课程环境名为 `ei-mnist`：

```
conda activate ei-mnist
```

确认 Python 能运行：

```
python --version
```

确认 PyTorch 能导入：

```
python -c "import torch; print(torch.__version__)"
```

确认 torchvision 能导入：

```
python -c "import torchvision; print(torchvision.__version__)"
```

确认 matplotlib 能导入：

```
python -c "import matplotlib; print(matplotlib.__version__)"
```

如果上述命令有任何一个报错，请先回到 Python 入门手册或环境搭建部分处理，不要直接进入训练代码。

4. 创建实验文件夹

建议本次实验使用单独文件夹：

Windows 示例：

```
mkdir D:\embodied_ai_mnist  
cd D:\embodied_ai_mnist
```

macOS/Linux 示例：

```
mkdir -p ~/embodied_ai_mnist  
cd ~/embodied_ai_mnist
```

最终目录结构会类似：

```
embodied_ai_mnist/  
mnist_mlp.py  
data/  
outputs/  
    training_log.csv  
    predictions.png  
mnist_mlp.pt
```

5. 实验原理：从图片到分类

5.1 MNIST 图片是什么样的

MNIST 中每张图片大小是：

28 x 28

它是灰度图，所以每个像素只有一个数值。可以把一张图片理解成一个 28 行、28 列的表格。

一张图片共有：

$28 * 28 = 784$

个像素。

5.2 为什么 MLP 要把图片展平

全连接网络接收的是一维向量，而不是二维图片。

所以我们会把：

1 x 28 x 28

变成：

784

这个操作叫 flatten，中文可以理解为“展平”。

示意：

原始图片：28 行 x 28 列
展平后：784 个数字排成一行

5.3 本次 MLP 网络结构

本次使用一个非常简单的 MLP：

输入层：784
隐藏层：128
输出层：10

解释：

层	含义
输入层 784	每张 MNIST 图片展平后的 784 个像素
隐藏层 128	模型学习到的中间特征
输出层 10	对应数字 0-9 的 10 个类别

模型输出不是直接输出一个数字，而是输出 10 个分数。哪个分数最大，模型就预测为哪个类别。

6. 完整代码

请新建文件：

mnist_mlp.py

把下面代码完整复制进去。

这份代码刻意写得比较直白，方便第一次接触深度学习的同学理解。代码里有大量注释，建议先整体跑通，再逐段阅读。

```
from pathlib import Path

import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# =====
# 1. 实验配置
# =====

# 每次训练取多少张图片。
# batch_size 越大，每一步处理的数据越多；初学阶段用 64 即可。
BATCH_SIZE = 64

# 训练轮数。
```

```

# 第一次实验建议先用 2，能较快看到结果。
EPOCHS = 2

# 学习率。
# 学习率控制模型参数每次更新的幅度。
LEARNING_RATE = 0.001

# MNIST 数据下载和保存的位置。
DATA_DIR = "./data"

# 实验结果保存的位置。
OUTPUT_DIR = Path("./outputs")


# =====
# 2. 选择运行设备
# =====

def get_device():
    """选择运行设备。

    如果电脑有可用 NVIDIA GPU，则使用 cuda。
    如果是 Apple Silicon Mac 且 MPS 可用，则使用 mps。
    否则使用 cpu。
    """
    if torch.cuda.is_available():
        return torch.device("cuda")

    if hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
        return torch.device("mps")

    return torch.device("cpu")


# =====
# 3. 准备数据
# =====

def build_dataloaders():
    """下载 MNIST 数据集，并创建 DataLoader。

    Dataset 负责保存数据。
    DataLoader 负责把数据按 batch 取出来。
    """

    # transforms.ToTensor() 会把图片转成 PyTorch tensor。
    # 对 MNIST 来说，原始图片会变成形状 [1, 28, 28] 的 tensor。
    transform = transforms.ToTensor()

    # train=True 表示训练集。
    train_dataset = datasets.MNIST(
        root=DATA_DIR,

```

```

        train=True,
        download=True,
        transform=transform,
    )

    # train=False 表示测试集。
    test_dataset = datasets.MNIST(
        root=DATA_DIR,
        train=False,
        download=True,
        transform=transform,
    )

    # shuffle=True 表示训练时打乱数据顺序。
    train_loader = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE,
        shuffle=True,
    )

    # 测试时不需要打乱顺序。
    test_loader = DataLoader(
        test_dataset,
        batch_size=BATCH_SIZE,
        shuffle=False,
    )

    return train_loader, test_loader

# =====
# 4. 定义 MLP 模型
# =====

class SimpleMLP(nn.Module):
    """一个最简单的全连接神经网络。

    输入: MNIST 图片, 形状为 [batch_size, 1, 28, 28]
    输出: 10 个类别分数, 形状为 [batch_size, 10]
    """

    def __init__(self):
        super().__init__()

        # nn.Flatten() 会把 [batch_size, 1, 28, 28]
        # 变成 [batch_size, 784]。
        self.flatten = nn.Flatten()

        # 这里定义真正的全连接网络。
        self.layers = nn.Sequential(
            nn.Linear(28 * 28, 128), # 第一层: 784 -> 128
            nn.ReLU(),               # 激活函数: 增加非线性表达能力

```

```

        nn.Linear(128, 10),      # 第二层: 128 -> 10
    )

def forward(self, x):
    # x 的原始形状: [batch_size, 1, 28, 28]
    x = self.flatten(x)

    # 展平后的形状: [batch_size, 784]
    scores = self.layers(x)

    # scores 的形状: [batch_size, 10]
    return scores

# =====
# 5. 训练一个 epoch
# =====

def train_one_epoch(model, train_loader, loss_fn, optimizer, device):
    """训练模型一轮。

    一轮 epoch 表示把训练集大致完整看一遍。
    """

    # 告诉 PyTorch: 现在是训练模式。
    model.train()

    total_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        # images 形状: [batch_size, 1, 28, 28]
        # labels 形状: [batch_size]

        # 把数据移动到指定设备, 例如 cpu / cuda / mps。
        images = images.to(device)
        labels = labels.to(device)

        # 第一步: 前向传播, 得到模型输出。
        outputs = model(images)

        # 第二步: 计算损失。
        loss = loss_fn(outputs, labels)

        # 第三步: 清空上一次的梯度。
        optimizer.zero_grad()

        # 第四步: 反向传播, 计算梯度。
        loss.backward()

        # 第五步: 更新模型参数。

```

```

optimizer.step()

# 下面几行用于统计平均 loss 和准确率。
batch_size = labels.size(0)
total += batch_size
total_loss += loss.item() * batch_size

# outputs.argmax(dim=1) 会取每一行最大分数的位置。
# 这个位置就是模型预测的数字类别。
predictions = outputs.argmax(dim=1)
correct += (predictions == labels).sum().item()

average_loss = total_loss / total
accuracy = correct / total

return average_loss, accuracy

# =====
# 6. 在测试集上评估
# =====

def evaluate(model, test_loader, loss_fn, device):
    """评估模型在测试集上的表现。"""

    # 告诉 PyTorch: 现在是评估模式。
    model.eval()

    total_loss = 0.0
    correct = 0
    total = 0

    # 测试阶段不需要计算梯度, 所以使用 torch.no_grad()。
    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = loss_fn(outputs, labels)

            batch_size = labels.size(0)
            total += batch_size
            total_loss += loss.item() * batch_size

            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()

    average_loss = total_loss / total
    accuracy = correct / total

    return average_loss, accuracy

```



```

# =====
# 7. 保存预测图片
# =====

def save_predictions_image(model, test_loader, device, output_path):
    """保存若干测试图片及其预测结果。"""

    model.eval()

    # 取出一个 batch 的测试数据。
    images, labels = next(iter(test_loader))

    # 把图片放到设备上, 送入模型。
    images_device = images.to(device)

    with torch.no_grad():
        outputs = model(images_device)
        predictions = outputs.argmax(dim=1).cpu()

    # 画前 10 张图片。
    plt.figure(figsize=(10, 4))

    for i in range(10):
        plt.subplot(2, 5, i + 1)

        # images[i] 的形状是 [1, 28, 28]。
        # squeeze() 去掉通道维度, 变成 [28, 28], 方便画图。
        image = images[i].squeeze()

        plt.imshow(image, cmap="gray")
        plt.title(f"pred={predictions[i].item()}, true={labels[i].item()}")
        plt.axis("off")

    plt.tight_layout()
    plt.savefig(output_path, dpi=150)
    plt.close()

# =====
# 8. 主函数
# =====

def main():
    # 创建输出文件夹。
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    # 选择设备。
    device = get_device()
    print("Using device:", device)

```

```

# 准备数据。
train_loader, test_loader = build_dataloaders()

# 创建模型，并移动到对应设备。
model = SimpleMLP().to(device)

# 损失函数。
# CrossEntropyLoss 适合多分类任务，例如 MNIST 的 10 分类。
loss_fn = nn.CrossEntropyLoss()

# 优化器。
# Adam 是常用优化器，初学阶段比较容易跑出稳定结果。
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

# 保存每一轮的日志。
log_rows = []

for epoch in range(1, EPOCHS + 1):
    train_loss, train_acc = train_one_epoch(
        model=model,
        train_loader=train_loader,
        loss_fn=loss_fn,
        optimizer=optimizer,
        device=device,
    )

    test_loss, test_acc = evaluate(
        model=model,
        test_loader=test_loader,
        loss_fn=loss_fn,
        device=device,
    )

    print(
        f"Epoch {epoch}: "
        f"train_loss={train_loss:.4f}, train_acc={train_acc:.4f}, "
        f"test_loss={test_loss:.4f}, test_acc={test_acc:.4f}"
    )

    log_rows.append(
        {
            "epoch": epoch,
            "train_loss": train_loss,
            "train_acc": train_acc,
            "test_loss": test_loss,
            "test_acc": test_acc,
        }
    )

# 保存模型参数。
model_path = OUTPUT_DIR / "mnist_mlp.pt"
torch.save(model.state_dict(), model_path)

```

```

print("Saved model:", model_path)

# 保存预测图片。
image_path = OUTPUT_DIR / "predictions.png"
save_predictions_image(model, test_loader, device, image_path)
print("Saved prediction image:", image_path)

# 保存训练日志。
log_path = OUTPUT_DIR / "training_log.csv"
with log_path.open("w", encoding="utf-8") as f:
    f.write("epoch,train_loss,train_acc,test_loss,test_acc\n")
    for row in log_rows:
        f.write(
            f"{row['epoch']},",
            f"{row['train_loss']:.6f},",
            f"{row['train_acc']:.6f},",
            f"{row['test_loss']:.6f},",
            f"{row['test_acc']:.6f}\n"
        )

print("Saved training log:", log_path)

if __name__ == "__main__":
    main()

```

7. 运行实验

7.1 MNIST 数据集下载方式

本实验使用 `torchvision.datasets.MNIST` 下载数据集。代码中这几行负责下载：

```

train_dataset = datasets.MNIST(
    root=DATA_DIR,
    train=True,
    download=True,
    transform=transform,
)

```

其中：

```
DATA_DIR = "./data"
```

表示数据会下载到当前实验文件夹下的 `data` 目录中。`download=True` 表示：如果本地没有 MNIST，就自动下载；如果已经下载过，就直接读取本地文件，不会重复下载。

第一次运行：

```
python mnist_mlp.py
```

程序会自动下载 MNIST。下载完成后，目录通常类似：

```
embodied_ai_mnist/  
  mnist_mlp.py  
  data/  
    MNIST/  
      raw/  
      processed/  
  outputs/
```

7.2 单独测试 MNIST 是否能下载

如果你想在正式训练前先测试数据集下载，可以新建一个临时文件：

```
download_mnist.py
```

写入：

```
from torchvision import datasets, transforms  
  
DATA_DIR = "./data"  
  
transform = transforms.ToTensor()  
  
train_dataset = datasets.MNIST(  
    root=DATA_DIR,  
    train=True,  
    download=True,  
    transform=transform,  
)  
  
test_dataset = datasets.MNIST(  
    root=DATA_DIR,  
    train=False,  
    download=True,  
    transform=transform,  
)  
  
print("训练集样本数：", len(train_dataset))  
print("测试集样本数：", len(test_dataset))  
  
image, label = train_dataset[0]  
print("第一张图片 shape：", image.shape)  
print("第一张图片标签：", label)  
print("MNIST 下载和读取成功。")
```

运行：

```
python download_mnist.py
```

如果看到类似输出：

```
训练集样本数： 60000
测试集样本数： 10000
第一张图片 shape: torch.Size([1, 28, 28])
第一张图片标签： 5
MNIST 下载和读取成功。
```

说明数据集已经准备好了。

7.3 如果课堂网络无法下载

如果现场网络无法下载 MNIST，推荐使用老师提前准备好的 `data` 文件夹。

老师可以提前在一台电脑上运行：

```
python download_mnist.py
```

确认下载成功后，把整个 `data` 文件夹发给学生或放到共享目录。学生只需要把 `data` 文件夹放到 `mnist_mlp.py` 同一级目录下：

```
embodied_ai_mnist/
  mnist_mlp.py
  data/
    MNIST/
      raw/
      processed/
```

然后正常运行：

```
python mnist_mlp.py
```

注意：不要只拷贝 `raw` 或只拷贝某几个文件。最省心的方式是直接拷贝整个 `data` 文件夹。

7.4 如何判断是不是已经下载过

如果 `data/MNIST` 已经存在，并且里面有 `raw`、`processed` 等内容，通常说明已经下载过。

也可以运行：

```
python download_mnist.py
```

如果很快输出训练集和测试集样本数，说明本地数据可用。

7.5 第一次运行正式实验

在 `mnist_mlp.py` 所在目录运行：

```
python mnist_mlp.py
```

第一次运行会自动下载 MNIST 数据集。下载完成后会开始训练。

你会看到类似输出：

```
Using device: cpu
Epoch 1: train_loss=0.xxxx, train_acc=0.xxxx, test_loss=0.xxxx, test_acc=0.xxxx
Epoch 2: train_loss=0.xxxx, train_acc=0.xxxx, test_loss=0.xxxx, test_acc=0.xxxx
Saved model: outputs/mnist_mlp.pt
Saved prediction image: outputs/predictions.png
Saved training log: outputs/training_log.csv
```

如果显示：

```
Using device: cuda
```

说明你的 NVIDIA GPU 被 PyTorch 检测到了。

如果显示：

```
Using device: mps
```

说明你的 Apple Silicon Mac 使用了 MPS。

如果显示：

```
Using device: cpu
```

也完全正常。本实验使用 CPU 就可以完成。

7.6 查看结果文件

训练结束后，查看 `outputs` 文件夹：

```
outputs/
  mnist_mlp.pt
  predictions.png
  training_log.csv
```

每个文件的含义：

文件	含义
<code>mnist_mlp.pt</code>	模型参数
<code>predictions.png</code>	模型预测示例图
<code>training_log.csv</code>	每一轮训练和测试结果

8. 逐段理解代码

8.1 import 部分

```
from pathlib import Path

import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

解释：

代码	作用
<code>Path</code>	创建文件夹、保存文件
<code>matplotlib.pyplot</code>	保存预测图片
<code>torch</code>	PyTorch 主库
<code>nn</code>	神经网络模块
<code>DataLoader</code>	按 batch 读取数据
<code>datasets</code>	下载和读取 MNIST
<code>transforms</code>	把图片转成 tensor

8.2 配置部分

```
BATCH_SIZE = 64
EPOCHS = 2
LEARNING_RATE = 0.001
```

这些叫超参数。它们不是模型自己学出来的，而是我们手动设置的。

参数	含义
<code>BATCH_SIZE</code>	每次训练取多少张图片
<code>EPOCHS</code>	训练几轮
<code>LEARNING_RATE</code>	学习率，控制参数更新幅度

8.3 DataLoader

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
)
```

这段代码表示：

1. 从训练集中取数据。
2. 每次取 `BATCH_SIZE` 张。
3. 每轮训练前打乱顺序。

训练时打乱顺序可以减少模型记住固定顺序的影响。

8.4 MLP 模型

```
self.layers = nn.Sequential(  
    nn.Linear(28 * 28, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10),  
)
```

解释：

1. `nn.Linear(28 * 28, 128)`：把 784 个输入变成 128 个隐藏特征。
2. `nn.ReLU()`：激活函数，让模型能学习非线性关系。
3. `nn.Linear(128, 10)`：输出 10 个数字类别的分数。

8.5 forward

```
def forward(self, x):  
    x = self.flatten(x)  
    scores = self.layers(x)  
    return scores
```

数据流动过程：


```
[batch_size, 1, 28, 28]
-> flatten
[batch_size, 784]
-> 全连接层
[batch_size, 128]
-> ReLU
[batch_size, 128]
-> 全连接层
[batch_size, 10]
```

8.6 训练五步

训练循环中最重要的是这五步：

```
outputs = model(images)
loss = loss_fn(outputs, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

解释：

步骤	含义
<code>outputs = model(images)</code>	模型做预测
<code>loss = loss_fn(outputs, labels)</code>	计算预测和真实答案的差距
<code>optimizer.zero_grad()</code>	清空旧梯度
<code>loss.backward()</code>	计算新梯度
<code>optimizer.step()</code>	更新模型参数

如果只记一件事，就记住这五步。这是深度学习训练代码的核心。

9. 实验记录表

请在实验完成后填写：

项目	记录
操作系统	
Python 版本	
PyTorch 版本	
使用设备	cpu / cuda / mps
epoch 数	2
batch size	64
learning rate	0.001
最终 train loss	
最终 train accuracy	
最终 test loss	
最终 test accuracy	
是否生成 <code>predictions.png</code>	是 / 否
遇到的问题	

10. 课堂检查问题

请尝试回答：

1. MNIST 每张图片为什么可以变成 784 个输入？
2. 本实验为什么不使用卷积神经网络？
3. `nn.Linear(28 * 28, 128)` 中的 `28 * 28` 是什么意思？
4. 输出层为什么是 10？
5. `ReLU` 的作用是什么？
6. `loss` 变小通常说明什么？
7. `accuracy` 代表什么？
8. 为什么训练集和测试集要分开？
9. 为什么测试时要用 `torch.no_grad()`？
10. 为什么模型和数据都要 `.to(device)`？

11. 常见问题

11.1 下载 MNIST 失败

可能是网络问题。解决办法：

1. 换网络后重试。
2. 使用老师提前下载好的 `data` 文件夹。
3. 确保 `data` 文件夹和 `mnist_mlp.py` 在同一目录下。

11.2 `ModuleNotFoundError: No module named 'torch'`

说明当前环境没有安装 PyTorch，或 VS Code 选错解释器。

先检查：

```
python -c "import sys; print(sys.executable)"
```

确认当前 Python 是否来自课程环境。

11.3 `ModuleNotFoundError: No module named 'torchvision'`

安装 torchvision：

```
python -m pip install torchvision
```

如果你是按 PyTorch 官方命令安装的，通常会同时安装 `torch` 和 `torchvision`。

11.4 训练很慢

CPU 训练慢是正常的。可以先保持 `EPOCHS = 1`。

```
EPOCHS = 1
```

本实验不以训练速度评分。

11.5 准确率很低

如果 test accuracy 很低，例如低于 0.5，请检查：

1. 是否把 `loss.backward()` 写漏。
2. 是否把 `optimizer.step()` 写漏。
3. 是否没有调用 `model.train()`。
4. 是否把 labels 和 predictions 比较写错。
5. 是否改动了模型但输入输出维度不正确。

11.6 维度报错

如果看到类似 shape、size、matrix multiplication 的报错，优先检查：

```
print(images.shape)
print(outputs.shape)
print(labels.shape)
```

本实验中期望：

```
images: [batch_size, 1, 28, 28]
outputs: [batch_size, 10]
labels: [batch_size]
```

12. 增量实验：给完成较快的同学

增量实验不是必须全部完成。建议至少选一个做，并记录修改前后的结果。

12.1 实验 A：修改隐藏层大小

找到：

```
nn.Linear(28 * 28, 128)
nn.Linear(128, 10)
```

把 128 改成 64：

```
nn.Linear(28 * 28, 64)
nn.ReLU()
nn.Linear(64, 10)
```

再改成 256：

```
nn.Linear(28 * 28, 256)
nn.ReLU()
nn.Linear(256, 10)
```

记录：

hidden size	test accuracy	训练速度感受
64		
128		
256		

思考：隐藏层越大，效果一定越好吗？

12.2 实验 B：修改学习率

找到：

```
LEARNING_RATE = 0.001
```

分别尝试：

```
LEARNING_RATE = 0.01
```

和：

```
LEARNING_RATE = 0.0001
```

记录：

learning rate	train loss	test accuracy
0.01		
0.001		
0.0001		

思考：

- 1. 学习率太大可能发生什么？
- 2. 学习率太小可能发生什么？

12.3 实验 C：增加一层隐藏层

原模型：

```
self.layers = nn.Sequential(  
    nn.Linear(28 * 28, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10),  
)
```

改成：

```
self.layers = nn.Sequential(  
    nn.Linear(28 * 28, 128),  
    nn.ReLU(),  
    nn.Linear(128, 64),  
    nn.ReLU(),  
    nn.Linear(64, 10),  
)
```

记录结果，并思考：

1. 增加一层后准确率是否提升？
2. 训练是否更慢？
3. 模型更复杂是否一定更好？

12.4 实验 D：把 ReLU 换成 Sigmoid

原模型：

```
nn.ReLU()
```

改成：

```
nn.Sigmoid()
```

记录结果：

激活函数	test accuracy
ReLU	
Sigmoid	

思考：不同激活函数对训练有什么影响？

12.5 实验 E：只看错误样本

在 `save_predictions_image` 中，现在画的是前 10 张测试图片。可以尝试修改代码，只保存预测错误的图片。

提示思路：

1. 得到 `predictions`。
2. 比较 `predictions` 和 `labels`。
3. 找出不相等的位置。
4. 画出这些图片。

这个实验稍难，适合已经能看懂主代码的同学。